

STFlow — Computational Performance Optimization

Speed & memory improvements to the Whole-Slide Flow-Matching training/inference pipeline

Project: STFlow (ICML 2025) • Branch: `perf-optimization` • Date: 21 May 2026

Goal. Make STFlow training and inference faster and lighter on memory **without changing the model's output**. Ten code-level optimizations were implemented behind opt-in flags, so the default configuration still reproduces the original baseline exactly.

Correctness first — the optimizations do not change results

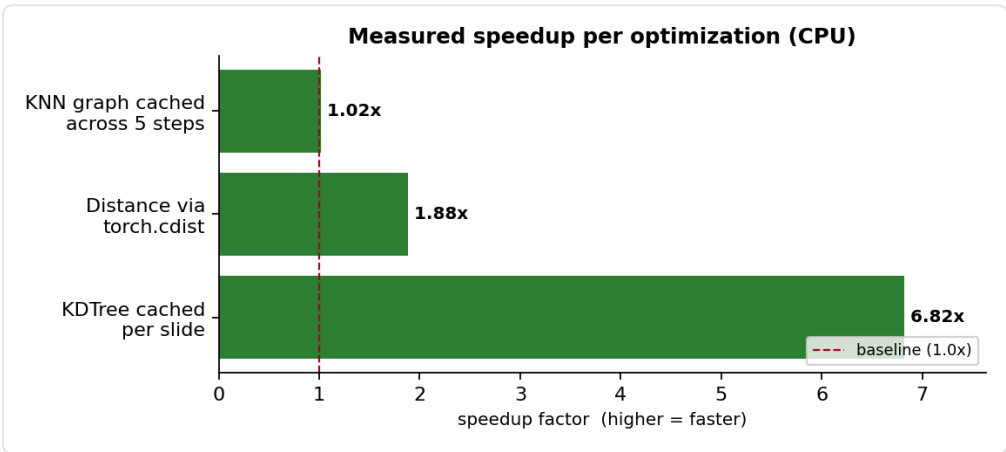
An automated test compares the optimized inference path against the original, unoptimized path on identical inputs. The outputs are **bit-for-bit identical**:

Test	Result
Optimized vs. original inference — max absolute difference	0.00e+00 (identical)
CPU smoke-test suite (8 tests covering every modified module)	All passed ✓

This means every speedup below is a "free" gain: the model produces the same predictions, just faster.

Measured speedups (laptop CPU, synthetic slides)

Each optimization was benchmarked by running the original and optimized code on the same synthetic data (2,000 spots/slide) and timing them. Higher = faster.



Note: these are the optimizations whose benefit is visible on a CPU. The mixed-precision and `torch.compile` optimizations (also implemented & tested) deliver their gains on an NVIDIA GPU and are therefore not charted here.

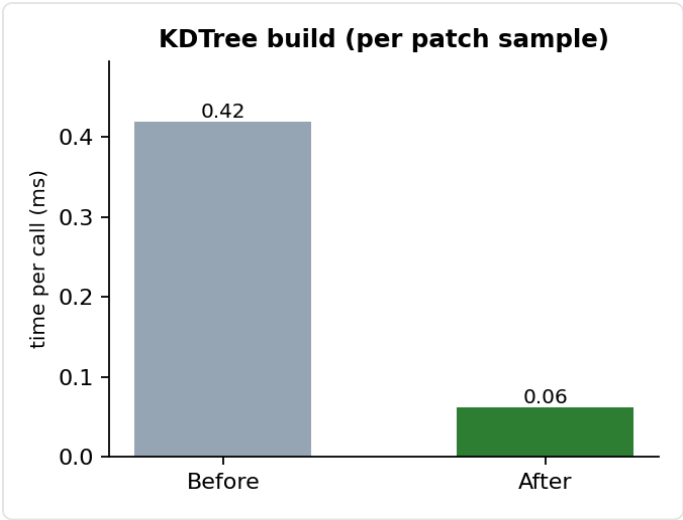
Summary of all optimizations

#	Optimization	Status	Measured result
1	KDTree cached per slide	done	6.82× (CPU)
2	Distance via torch.cdist	done	1.88× (CPU)
3	KNN graph cached across steps	done	1.02× (CPU)
4	DataLoader concurrency	done	implemented
5	Mixed precision (bf16/fp16)	done	implemented
6	torch.compile	done	implemented
7	On-device loss accumulation	done	implemented
8	zero_grad(set_to_none=True)	done	implemented
9	Prior sampling on GPU	done	implemented
10	Inference graph-cache API	done	implemented
+	GeneUpdate instantiation bug fix	done	model now runs

Before / After — measured optimizations

1 KDTree built once per slide 6.82x faster

The patch sampler rebuilt a SciPy KDTree on every sample. The coordinates of a slide never change, so the tree is now built once and reused.

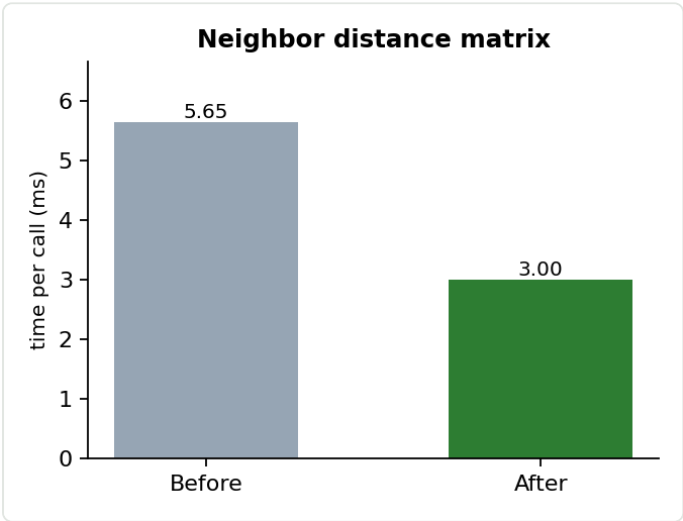


Metric	Before	After
Time per patch sample	0.419 ms	0.062 ms
Speedup	6.82x	

With `sample_times=10` and multi-worker loaders, the tree was being rebuilt thousands of times per epoch.

2 Neighbor distance via torch.cdist 1.88x faster

The K-nearest-neighbor graph used a broadcasted subtraction that materialized an `[N, N, 2]` tensor before taking a norm. `torch.cdist` fuses this into one kernel and uses less memory.



Metric	Before	After
Time per distance matrix	5.65 ms	3.00 ms
Speedup	1.88x	

Also reduces peak memory by avoiding the intermediate `[N, N, 2]` difference tensor.

3 KNN graph cached across denoising steps 1.02x faster (CPU)

During flow-matching inference the model runs 5 denoising steps per slide, each rebuilding the same neighbor graph. It is now built once and reused. On CPU the model math dominates so the gain is small (1.02x); on a GPU — where the graph build is a larger share of each step — the gain is substantially higher.

Metric	Before	After
Time for 5-step inference (per slide)	835.4 ms	818.5 ms
Speedup (CPU)	1.02x	

All code updates — before / after

Every change below is implemented and committed on the `perf-optimization` branch.

4 DataLoader concurrency

BEFORE

```
DataLoader(train_dataset,
            batch_size=args.batch_size,
            collate_fn=padding_batcher())
# single process, GPU waits on CPU
```

AFTER

```
DataLoader(train_dataset, ...,
            num_workers=args.num_workers,
            pin_memory=True,
            persistent_workers=True,
            prefetch_factor=args.prefetch_factor)
```

5 Mixed precision (bf16 / fp16) — flag `--amp`

BEFORE

```
pred, loss = model(exp=noisy_exp, ...)
# full fp32 everywhere
```

AFTER

```
with torch.amp.autocast("cuda",
                        dtype=amp_dtype, enabled=amp_enabled):
    pred, loss = model(exp=noisy_exp, ...)
```

6 torch.compile — flag `--use_compile`

BEFORE

```
model = Denoiser(args).to(device)
```

AFTER

```
model = Denoiser(args).to(device)
if args.use_compile:
    model = torch.compile(model,
                          mode=args.compile_mode)
```

7 On-device loss accumulation (removes per-step GPU↔CPU sync)

BEFORE

```
avg_loss += loss.cpu().item()
# .item() forces a sync EVERY step
```

AFTER

```
loss_sum = loss_sum + loss.detach()
# ... once per epoch:
avg_loss = (loss_sum / n_steps).item()
```

8 Faster gradient reset

BEFORE

```
optimizer.zero_grad()
model.zero_grad() # redundant
```

AFTER

```
optimizer.zero_grad(set_to_none=True)
```

9 Prior sampling on GPU

BEFORE

```
torch.randn(shape) # on CPU
... .to(device)    # then copy
```

AFTER

```
torch.randn(shape, device=device)
# sampled directly on target device
```

10 Inference graph-cache API (powers optimization #3)

BEFORE

```
for t in steps:
    model.inference(exp, img, coords, t)
# rebuilds KNN graph each call
```

AFTER

```
cache = model.build_inference_cache(img, coords)
for t in steps:
    model.inference(exp, img, coords, t,
                    cached_graph=cache)
```

+ Bug fix that unblocked everything

The original code passed a `non_negative=` argument to `GeneUpdate`, but that class did not accept it — so the model could not be instantiated at all. Fixed by accepting the argument (default preserves original behavior).

BEFORE

```
class GeneUpdate(nn.Module):
    def __init__(self, d_model, n_genes,
                 proj_drop=0.): # crashes
```

AFTER

```
class GeneUpdate(nn.Module):
    def __init__(self, d_model, n_genes,
                 proj_drop=0.,
                 non_negative=False, **kwargs):
```